ESP32 Logic Analyzer: Project Handoff

Audience: the teammate who will build the PC/desktop app. You will use Claude to help, so a ready-to-paste prompt is at the end (section 11).

1. What this project is

An 8-channel logic analyzer built on an ESP32.

- The ESP32 samples 8 digital inputs at **5 MHz** using the I2S peripheral + DMA (no CPU per sample).
- It compresses the samples with **run-length encoding (RLE)**, wraps them in **packets** with a CRC, and sends them over **WiFi (TCP)** to a laptop.
- A reference Python decoder (`la_decode.py`) already reads the stream and can write a VCD waveform file.
- **Your job: the app that shows the graph and other stats using these packets.** Everything on the ESP32 side is finished and tested.

```
8 input pins -> I2S0 (slave, parallel/camera mode) -> DMA ring (8 x 4000 B)
  -> RLE encoder -> packetizer (header + CRC16) -> message buffer
  -> TCP server on the ESP32 (port 5000)  ==WiFi==>  YOUR APP
```

2. Who does what

Done (Aayu, firmware + protocol)	Yours (the app)
I2S + DMA capture at 5 MHz, 8 channels	Connect to the ESP32 over TCP
RLE compression	Parse frames (resync, CRC check)
Packet framing + CRC16, sequence numbers	Rebuild the timeline from RLE events
Gap / loss reporting (`lost_before`, `seq`)	Waveform viewer (zoom, pan, channel names)
STATS health frame every second	Stats panel, measurements, export
WiFi TCP transport, reconnect handling	Handle gaps, restarts, reconnects in the UI
Reference decoder `la_decode.py` + VCD output	Optional: triggers, protocol decoders

You do not need to touch the firmware. Treat the packet format (section 5) as a fixed contract.

3. Folder layout

Path	What it is
`src/main.cpp`	setup(): starts transport, clocks, I2S, DMA ring
`src/i2s_comm.cpp`	Pin routing, I2S config, sample clock (LEDC) and 10 kHz test signal
`src/dma_ring.cpp`	DMA descriptors, ISR, consumer task, overrun detection
`src/compress.cpp`	RLE encoder (also has the varint helpers)
`src/packet.cpp`	**Packet framing, CRC16, loss accounting (the "packaging" part)**
`src/stream.cpp`	Glue: samples -> RLE -> packets -> transport; heartbeat flush; STATS timing
`src/transport_wifi.cpp`	WiFi TCP server (or its own access point)
`src/transport.cpp`	UART transport (only compiled when `USE_WIFI` is 0)
`include/config.h`	All settings (pins, sample rate, WiFi)
`include/packet.h`	**The wire format, documented in comments**
`la_decode.py`	**Reference decoder; port this logic to your app**
`tools/host_test/`	PC-only tests for packet + RLE (no ESP32 needed)
`tools/sample_stream.bin`	A recorded stream with deliberate gaps, for offline app testing

4. Running it (to get live data)

Hardware self-test needs two jumpers only:

- GPIO22 -> GPIO23 (sample clock loop)
- GPIO2 -> GPIO21 (10 kHz test square wave into CH0)

Real channels: CH0..CH7 = GPIO 21, 19, 18, 5, 4, 15, 13, 12. Common GND. **3.3 V only.** GPIO12 and GPIO15 are strapping pins: do not drive GPIO12 high at power-up.

Steps:

1. In `include/config.h` set `WIFI_SSID` and `WIFI_PASS` (2.4 GHz network). Do not commit your password.
2. `pio run -t upload` (PlatformIO, board esp32dev, Arduino framework).
3. Open the serial monitor at 115200. Read: `[wifi] on the laptop: python la_decode.py tcp://192.168.x.y:5000`. The IP can change after a reset.
4. Laptop on the same WiFi: `python la_decode.py tcp://192.168.x.y:5000 --vcd out.vcd` (needs Python; no extra packages for TCP).
5. Healthy output once per second: `lost_buf=0 tx_dropped=0 crc_bad=0 lost_total=0`.

Alternative: set `WIFI_AP_MODE 1` and the ESP32 creates its own network (`ESP32-LogicAnalyzer` / `logic1234`); the app then connects to `192.168.4.1:5000`.

No hardware? Run `python la_decode.py tools/sample_stream.bin` to decode a recorded stream. Your app can read the same file as a test input (it contains deliberate gaps).

5. The packet format (the "packaging" part)

The TCP connection is a plain byte stream of back-to-back frames. All multi-byte fields are **little-endian**. There is no text on the stream in WiFi mode.

5.1 Frame

Offset	Size	Field	Meaning
0	1	sync0	`0xA5`
1	1	sync1	`0x5A`
2	1	type	`1` = DATA, `2` = STATS
3	1	flags	bit0 (`0x01`) = GAP: samples were lost right before this packet
4	2	seq	increments by 1 per frame built (wraps at 65535)
6	2	len	payload length in bytes (max 12 + 1024 = 1036)
8	len	payload	depends on type
8+len	2	crc16	CRC-16/CCITT-FALSE (poly `0x1021`, init `0xFFFF`, no reflection) over bytes **[2 .. end of payload]** (type, flags, seq, len, payload; **not** the two sync bytes)

Total frame size = 8 + len + 2. Maximum frame = 1046 bytes.

5.2 DATA payload (type 1)

Offset	Size	Field
0	8	`start_sample` u64: index (since the board started) of the first sample of the first event
8	4	`lost_before` u32: samples lost between the previous packet and this one (already counted inside `start_sample`)
12	n	RLE events, whole events only

Time of sample `i` = `i / sample_hz` seconds (5 MHz -> 200 ns per sample). `sample_hz` comes from the STATS frame (default 5,000,000).

5.3 RLE event

```
[state byte][run length as varint]
```

- **State byte**: bit `i` is the level of channel `CHi` (bit0 = CH0 = GPIO21 ... bit7 = CH7 = GPIO12).
- **Run length**: how many consecutive samples held that state.
- **Varint**: 7 bits per byte, little-endian, top bit set means another byte follows (max 4 bytes, max run 0x0FFFFFFF).
- Samples of an event cover indices `pos .. pos+run-1`, then `pos += run`.
- Two neighbouring events can have the **same** state (the firmware flushes a pending run every 20 ms as a heartbeat, and splits very long runs). Merge them when drawing.

5.4 STATS payload (type 2, 30 bytes, sent about once per second)

Offset	Size	Field
0	1	version (1)
1	1	channels (8)
2	4	sample_hz
6	4	produced (DMA buffers filled)
10	4	consumed (buffers processed)
14	4	lost_buffers (DMA overruns on the ESP32)
18	4	max_proc_us (worst processing time per buffer)
22	4	tx_drop_pkts (packets the ESP32 could not queue)
26	4	tx_drop_samples (samples inside those packets)

5.5 Worked example (generated by the real firmware code)

DATA frame, events: state `0x01` for 3 samples, then state `0x00` for 200 samples (`200` encodes as varint `C8 01`):

```
A5 5A 01 00 00 00 11 00   header: type=1 flags=0 seq=0 len=17
00 00 00 00 00 00 00 00   start_sample = 0
00 00 00 00               lost_before  = 0
01 03                     state 0x01, run 3
00 C8 01                  state 0x00, run 200
4E F3                     CRC16 = 0xF34E (stored little-endian)
```

STATS frame (sample_hz=5,000,000, 8 ch, produced=100, consumed=99, max_proc_us=646):

```
A5 5A 02 00 01 00 1E 00 01 08 40 4B 4C 00 64 00 00 00 63 00 00 00 00 00 00 00 86 02 00 00 00 00
  00 00 00 00 00 00 19 27
```

Both frames decode with `la_decode.py` with `crc_bad=0`.

6. Receiver rules (what your app must do)

1. **Resync.** Search the buffer for `A5 5A`. Read `len`; if `len` > 1036 it is not a real header, so skip one byte and search again. Wait until the full frame has arrived.
2. **CRC.** Compute CRC16 over bytes `[2 .. 8+len)`. On mismatch, count `crc_bad`, drop one byte, resync. (Over WiFi/TCP this should be zero; the CRC matters for UART and for safety.)
3. **Sequence.** If `seq != expected`, frames were lost; report how many.
4. **Join mid-run.** The board keeps counting while no client is connected. The first DATA frame you see sets the baseline: `pos = start_sample - lost_before`. Do not treat earlier samples as lost.
5. **Gaps.** Before applying a DATA frame, if `start_sample != pos + lost_before`, extra samples are missing (a lost frame). Total gap = `lost_before` + that extra. Draw the gap as **unknown** (shown as `x` in the VCD) and never invent data. Then set `pos = start_sample` and walk the events.
6. **Board restart.** If `start_sample < pos` the board rebooted and the counter went back to 0. Start a new capture (or continue the timeline with an offset, as the decoder does).
7. **Reconnect.** If the TCP connection drops, retry every couple of seconds and re-baseline on the next frame.
8. **Idle line.** A line that never changes still produces data about every 20 ms because of the heartbeat flush, so silence for much longer than that means something is wrong.

Reference implementation: `la_decode.py` (`Decoder.feed` and `Decoder._frame`).

7. Limits you should know

- 8 channels, 5 MHz sample rate, 3.3 V inputs.
- WiFi bandwidth is the real limit. A 10 kHz signal is tiny (about 100 frames per second); fast, busy signals produce far more RLE data. Watch `tx_dropped` in the STATS frame: if it rises, WiFi cannot keep up and the app will see gaps.
- The TCP server accepts one client at a time.
- The firmware does not trigger. It streams continuously; any trigger logic has to be done in the app on the decoded stream.
- `out.vcd` files grow fast (about 16 MB for a short run). Do not leave VCD writing on for long.
- Capture index is a 64-bit sample counter. At 5 MHz it never overflows in practice.

8. Your work: suggested plan

Do these in order; each step is usable on its own.

1. **Parser.** TCP client + frame parser + CRC, with `tools/sample_stream.bin` as the first test input. Match the `la_decode.py` output (frames, gaps, `lost_total`).
2. **Timeline model.** Turn events into per-channel edge lists (time, level). Handle gaps as unknown spans, restarts, and merging of equal neighbouring events.
3. **Waveform view.** 8 stacked digital traces, time axis in ns/us/ms, zoom and pan, gaps drawn visibly, live scrolling mode.
4. **Stats panel.** Live values from STATS frames: sample rate, buffers produced/consumed, lost buffers, dropped packets, link quality (`crc_bad`, missing frames), connection state.
5. **Measurements.** Per channel: frequency, period, duty cycle, pulse width, edge count; cursors with delta-time readout.
6. **Export and save.** VCD (PulseView / GTKWave compatible), CSV of edges, save and replay of the raw stream.
7. **Optional.** Software trigger on the decoded stream, channel naming, protocol decoders (UART, I2C, SPI) on top of the edge lists.

Any language is fine (Python + PyQtGraph / Matplotlib, a web app with a small server, etc.). Python is the easiest start because the reference decoder is already Python.

9. Quick checklist before you start

- [ ] Self-test works: CH0 shows a 10 kHz square wave (period 100 us, 50 us per half).
- [ ] `la_decode.py tools/sample_stream.bin` prints gaps and `crc_bad 0`.
- [ ] Your parser reproduces the numbers above.
- [ ] You can connect to the live board and see about 100 DATA frames per second with the self-test.

10. Troubleshooting

Symptom	Likely cause
Cannot connect (timeout, WinError 10065/10051)	Laptop is not on the same network as the ESP32, or the router has client isolation on
IP changed	The ESP32 got a new address after a reset; read the serial monitor line again
`connect timeout` on the serial monitor	Wrong SSID/password or a 5 GHz-only network
`tx_dropped` rising	WiFi too slow for the signal rate; fewer/slower signals or move closer to the router
`[board] restarted`	Board reset; the serial monitor prints `reset_reason=N` at boot (1 power-on, 3 software, 4 panic, 5/6/7 watchdog, 9 brownout)
CH3 / CH5 read high with nothing attached	GPIO5 and GPIO15 have pull-ups at boot; connect real signals

11. Prompt to paste into Claude

Attach (or paste) this whole zip's key files: `HANDOFF.md`, `include/packet.h`, `src/packet.cpp`, `src/compress.cpp`, `la_decode.py`, and `tools/sample_stream.bin`. Then paste the prompt below.

```
You are helping me build the desktop/PC application for an ESP32-based 8-channel logic analyzer.
   The firmware is finished; my job is the app. Read the attached HANDOFF.md first, then
  include/packet.h, src/packet.cpp, src/compress.cpp and la_decode.py, and treat the packet
  format in HANDOFF.md section 5 as a fixed contract.

First, in your own words, explain back to me:
1. What the ESP32 side does and what is already done.
2. The exact frame format: sync bytes, header, DATA and STATS payloads, the RLE event and varint
   encoding, and how the CRC is computed.
3. How gaps, lost frames, restarts, joining mid-stream and reconnects must be handled by the
  receiver.
4. What my part is (the app) and the plan in section 8.
Keep it short and correct, and point out anything in the files that looks ambiguous or
  inconsistent.

Then help me build the app step by step, following section 8 in order:
- Ask me ONE question at the start about my preferred language and GUI framework (suggest Python
   with PyQtGraph if I have no preference), and my operating system.
- Step 1: a frame parser + TCP client that reproduces the output of la_decode.py on
  tools/sample_stream.bin. Write unit tests that decode the two worked-example frames in
  HANDOFF.md section 5.5 and the sample stream (expected: 140 frames, 3,352,000 samples, 152,000
   lost, 0 CRC errors).
- Then the timeline model, waveform view (zoom, pan, gaps shown as unknown), live stats panel,
  measurements, and export (VCD and CSV).
- After each step, tell me how to run and test it, and wait for me before moving on.
- Never invent data for gaps; mark them unknown. Never change the packet format. Do not modify
  the firmware unless I ask.
- If something in the stream looks wrong (CRC failures, sequence gaps, rising tx_drop counters),
   help me diagnose it using the STATS fields before changing code.

Connection details: the ESP32 prints "[wifi] on the laptop: ... tcp://<ip>:5000" on its serial
  monitor at 115200 baud. The app connects to that IP on port 5000 as a TCP client. Sample rate
  is 5 MHz (200 ns per sample), channels are bits 0..7 of the state byte.
```